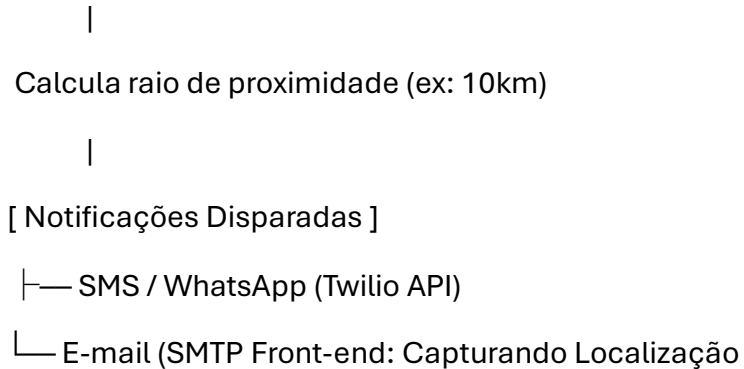


Para que o **OvniVra** monitore sua localização em tempo real e envie alertas por SMS, WhatsApp e E-mail, você precisa conectar três camadas principais: a **coleta de geolocalização no front-end** (app), o **cálculo de distância no back-end** (FastAPI) e a **integração dos serviços de mensageria**.

Architecture do Fluxo

-> Envia GPS (Lat/Lng) em Stream -> [Back-end FastAPI]



Continuamente

No seu aplicativo (como Flutter ou React Native), você deve escutar as atualizações do sensor GPS do dispositivo e enviar a coordenada atualizada para a API.

Exemplo em Flutter (geolocator):

SendGrid)

```
Import 'package:geolocator/geolocator.dart';
```

```
Import 'package:http/http.dart' as http;
```

```
Import 'dart:convert';
```

```
Void iniciarMonitoramentoGPS(String userId) {
```

```
  // Configura atualização por distância ou tempo
```

```
  LocationSettings locationSettings = const LocationSettings(
```

```
    Accuracy: LocationAccuracy.high,
```

```
    distanceFilter: 100, // Notifica a cada 100 metros percorridos
```

```
);
```

```

Geolocator.getPositionStream(locationSettings: locationSettings).listen((Position
position) {

    // Envia localização atualizada para o backend

    http.post(

        Uri.parse('https://api.ovnivra.com/usuario/localizacao'),

        Headers: {'Content-Type': 'application/json'},

        Body: jsonEncode({

            'usuario_id': usuarioid,

            'latitude': position.latitude,

            'longitude': position.longitude,

        }),

    );

});

}

```

Back-end: Cálculo da Proximidade e Alerta (FastAPI + Python)

No back-end em Python, você precisa de uma função para calcular a distância física entre o usuário e o avistamento relatado usando a fórmula de Haversine:

```

Import math

```

```

From fastapi import FastAPI, BackgroundTasks

```

```

From pydantic import BaseModel

```

```

App = FastAPI()

```

```

Def calcular_distancia_km(lat1, lon1, lat2, lon2):

```

```

    R = 6371.0 # Raio da Terra em km

```

```

    Dlat = math.radians(lat2 - lat1)

```

```

    Dlon = math.radians(lon2 - lon1)

```

```

    A = math.sin(dlat / 2)**2 + math.cos(math.radians(lat1)) *
    math.cos(math.radians(lat2)) * math.sin(dlon / 2)**2

```

```
C = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
```

```
Return R * c
```

```
# Exemplo de verificação ao registrar um novo OVNI
```

```
Def verificar_e_notificar_proximidade(ovni_lat, ovni_lng, usuario):
```

```
    Distancia = calcular_distancia_km(usuario.lat, usuario.lng, ovni_lat, ovni_lng)
```

```
    RAIO_ALERTA_KM = 10.0 # Notifica se estiver a menos de 10km
```

```
    If distancia <= RAIO_ALERTA_KM:
```

```
        Mensagem = f" ⚠️ ALERTA OVNI! Objeto não identificado detectado a  
{distancia:.1f}km da sua localização!"
```

```
        # Dispara alertas em background
```

```
        Enviar_sms(usuario.telefone, mensagem)
```

```
        Enviar_whatsapp(usuario.telefone, mensagem)
```

```
        Enviar_email(usuario.email, "Alerta de OVNI Próximo!", mensagem)
```

```
Configurando os Canais de Envio
```

```
A. SMS e WhatsApp (via Twilio API)
```

A API do Twilio é o padrão mais prático para unificar envios de SMS e mensagens diretas no WhatsApp.

```
From twilio.rest import Client
```

```
TWILIO_ACCOUNT_SID = "seu_sid"
```

```
TWILIO_AUTH_TOKEN = "seu_token"
```

```
Client = Client(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)
```

```
Def enviar_sms(numero_destino: str, texto: str):
```

```
    Client.messages.create(
```

```
Body=texto,  
From_='+1234567890', # Seu número Twilio  
To=numero_destino  
)
```

Def enviar_whatsapp(numero_destino: str, texto: str):

```
Client.messages.create(  
    Body=texto,  
    From_='whatsapp:+14155238886', # Sandbox ou número empresarial do Twilio  
    To=f'whatsapp:{numero_destino}'  
)
```

B. E-mail (via FastAPI BackgroundTasks / SMTP)

Para não travar o servidor durante a entrega do e-mail, execute o processo de envio de forma assíncrona ou em background:

```
Import smtplib
```

```
From email.mime.text import MIMEText
```

Def enviar_email(destino: str, assunto: str, corpo: str):

```
Msg = MIMEText(corpo)  
Msg['Subject'] = assunto  
Msg['From'] = alertas@ovnivra.com  
Msg['To'] = destino
```

With smtplib.SMTP_SSL('smtp.gmail.com', 465) as server:

```
Server.login(seu\_email@gmail.com, "sua_senha_de_app")  
Server.sendmail(alertas@ovnivra.com, destino, msg.as_string())
```

Considerações Técnicas Importantes

Permissões em Segundo Plano (Background Location):

Tanto Android quanto iOS exigem que o usuário autorize explicitamente a opção “Permitir o tempo todo” nas configurações de GPS para que o aplicativo continue reportando a localização quando a tela estiver desligada.

Otimização de Bateria e Banco de Dados:

Em vez de salvar cada alteração de coordenadas no banco de dados SQLite/PostgreSQL, mantenha a última localização ativa do usuário em memória em uma instância Redis (usando a extensão Geospatial nativa do Redis GEOADD / GEORADIUS).

Custos de Mensageria:

Os envios de SMS e WhatsApp através de gateways como Twilio ou Z-API possuem custo por mensagem enviada. Considere adicionar uma opção dentro do OvniVra para o usuário escolher se prefere alertas urgentes só por Push Notification (gratuito via Firebase Cloud Messaging) ou por SMS/WhatsApp.